SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Programme: **B.E / B.Tech**

Course Code: **CSA3714** Course Name: **SOFTWARE TESTING**

Submission Date: 16-11-2026 Date of Issue: 6-11-2026

Duration: **1 Week** Max Marks: **50 Marks**

ASSIGNMENT – 3

CO	Course Outcome Statement
CO3	Analyse unit, integration, system, regression, and acceptance testing for validating software systems.

Assignment Title: Evaluate and Defend the Levels of Testing for a Farmer-to-Buyer Marketplace

Scenario — Farmer-to-Buyer Marketplace (AgriMart)

A state agriculture department is funding AgriMart, a mobile marketplace where farmers list their produce and buyers purchase directly. The app must work on low-cost phones, in regional languages, and on slow rural networks. About 25,000 farmers and 3,000 buyers are expected in the first season.

You are the Test Manager. You must decide how AgriMart will be tested at the unit, integration, system and acceptance levels, and defend your decisions to the department.

AgriMart modules:

ID	Module	Depends on	Status in Week 4
M1	**Registration and KYC**	**Aadhaar e-KYC service (external)**	**Complete**

M2	**Produce Listing**	**M1**	**Complete**
M3	**Price Engine**	**Market-price API (external), M2**	**Complete**
M4	**Cart and Order**	**M2, M3**	**In progress**
M5	**Payment (UPI gateway)**	**UPI gateway (external), M4**	**Not started**
M6	**Logistics and Pickup**	**M4, transport partner API (external)**	**Not started**
M7	**Notifications (SMS, voice)**	**M1–M6**	**In progress**

Price Engine (M3) rule:

Suggested price = today's market price × quality factor (Grade A 1.10, Grade B 1.00, Grade C 0.85). If the market-price API does not respond within 3 seconds, the last available price (not older than 24 hours) is used; otherwise listing is blocked. Farmers may set their own price within ±15 % of the suggested price.

Changes made in the last two weeks:

C#	Change
C1	**Quality factor for Grade C changed from 0.80 to 0.85 (M3).**
C2	**Order cancellation window reduced from 2 hours to 1 hour (M4).**
C3	**New Tamil voice prompts added (M7).**

C4	**Fix for duplicate farmer registration with the same mobile number (M1).**

Design Challenge:

The department now requires integration with the national e-NAM trading platform and a hotfix release in one week.

Re-evaluate your testing strategy and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Scope of Unit Testing 5 Marks

CO3 | Bloom's Level: K5 – Evaluate

The developers want to treat each screen of the app as a "unit".

What should count as a unit in AgriMart, and does the choice affect the defects that unit testing can find?

Your answer must include:

- **A definition of a unit for AgriMart, with examples from M3 and M4**
- **The goals of unit testing for these units**
- **The defects that unit testing can and cannot find**
- **A comparison of "screen as unit" with "function/class as unit"**
- **Your recommended unit definition, with justification**

You are expected to decide and defend the unit definition.

Evidence Expected

A justified definition of units for AgriMart with an evaluation of what unit testing can achieve.

TASK 2 – Evaluate Unit Test Planning **2 Marks**

CO3 | Bloom's Level: K5 – Evaluate

M3 is complete and must be unit-tested this week.

What must a unit test plan for the Price Engine contain to be useful?

Discuss:

- **The objectives, features to be tested, approach and pass/fail criteria for M3**
- **The resources and time needed**
- **Why unit test planning is often skipped, and the risk of skipping it here**

Your plan must be specific to M3.

Evidence Expected

A focused unit test plan for M3 with justified contents.

TASK 3 – Design and Evaluate Unit Tests for the Price Engine **6 Marks**

CO3 | Bloom's Level: K5 – Evaluate

Design unit tests for the Price Engine rule.

Which unit tests give confidence that no farmer is shown a wrong or unfair price?

Discuss:

- **Test cases for each quality grade, including boundaries of the ±15 % limit**
- **Test cases for API response within 3 seconds, after 3 seconds, and with a price older than 24 hours**
- **Expected results calculated from the rule**
- **Positive, negative and exception tests**
- **Which tests must be updated because of change C1**

- **How you would judge whether your unit test set is sufficient**

You must show expected results and evaluate the sufficiency of your tests.

Evidence Expected

A complete unit test design for M3 with expected results and a justified judgement on its sufficiency.

TASK 4 – Design and Evaluate the Test Harness 4 Marks

CO3 | Bloom's Level: K5 – Evaluate

M3 depends on an external market-price API and on M2.

How can M3 be tested repeatably when the real API changes every day and sometimes fails?

Discuss:

- **The drivers and stubs needed to test M3, and what each must simulate**
- **How the stub will simulate slow responses, failures and old prices**
- **How test data and expected results will be stored for re-use**
- **The risks of relying on stubs, and how they can be reduced**

Your harness must be justified for M3, not shown as a generic diagram.

Evidence Expected

A test harness design for M3 with a justified evaluation of its benefits and limitations.

TASK 5 – Evaluate Integration Strategies 3 Marks

CO3 | Bloom's Level: K5 – Evaluate

Modules are at different stages of completion and three external services are involved.

Which integration strategy finds the most serious interface defects earliest, given the current status of modules?

Discuss:

- **Big-bang, top-down, bottom-up and sandwich integration applied to AgriMart**
- **How the module status table affects each strategy**
- **How external services should be handled during integration**

You are expected to evaluate the strategies for this system, not describe them generically.

Evidence Expected

A comparative evaluation of integration strategies for AgriMart with a justified choice.

TASK 6 – Construct and Evaluate the Integration Test Plan 3 Marks

CO3 | Bloom's Level: K5 – Evaluate

Prepare an integration test plan for the strategy you selected.

In which order should the modules be joined, and what must be proved at each step?

Discuss:

- **The integration order and the stubs/drivers needed at each step**
- **Interface test cases for M3–M4 and M4–M5**
- **Entry and exit criteria for each integration step**

Your plan must be consistent with the module status table.

Evidence Expected

A justified integration test plan with order, interface tests and criteria.

TASK 7 – Evaluate System Testing Needs **4 Marks**

CO3 | Bloom's Level: K5 – Evaluate

AgriMart will be used on low-cost phones by farmers with limited literacy, on slow networks.

Which kinds of system testing are essential for AgriMart, and which can safely be reduced?

Discuss:

- **Functional, performance/load, usability, security, compatibility and recovery testing for AgriMart**
- **The specific risk each type addresses in this system**
- **Test scenarios for at least three system test types**
- **The types you would reduce if time is short, and the risk of doing so**

You must prioritise and justify the system test types.

Evidence Expected

A prioritised and justified system testing approach with representative scenarios.

TASK 8 – Evaluate Regression Testing **4 Marks**

CO3 | Bloom's Level: K5 – Evaluate

Four changes were made in the last two weeks.

How much of AgriMart must be re-tested after these changes, and how do you decide?

Discuss:

- **Retest-all, selective and risk-based regression testing**
- **The regression tests needed for each change C1–C4, and why**
- **Modules indirectly affected by each change**

- **How automation could support regression testing in AgriMart**

Your regression selection must be traced to the changes.

Evidence Expected

A justified regression test selection for C1–C4 with reasoning on indirect impact.

TASK 9 – Evaluate Alpha and Beta Testing **3 Marks**

CO3 | Bloom's Level: K5 – Evaluate

The department wants a beta test with real farmers before the season.

Who should take part in alpha and beta testing of AgriMart, and what can these tests reveal that earlier testing cannot?

Discuss:

- **The purpose and setting of alpha and beta testing**
- **The users, locations and duration you would select for each**
- **How feedback will be collected from farmers with limited literacy**

Your plan must be justified for the AgriMart user base.

Evidence Expected

A justified alpha and beta testing plan suited to AgriMart users.

TASK 10 – Evaluate Acceptance Testing **4 Marks**

CO3 | Bloom's Level: K5 – Evaluate

The agriculture department must formally accept AgriMart before payment to the vendor.

What evidence should the department demand before accepting AgriMart?

Discuss:

- **Acceptance criteria that are measurable and agreed in advance**
- **Acceptance test scenarios covering farmers, buyers and administrators**
- **Who should perform the acceptance tests and who should sign off**
- **What should happen if some criteria are not met**

Your criteria must be specific and measurable.

Evidence Expected

Measurable acceptance criteria and scenarios with a justified sign-off process.

TASK 11 – Recommend and Defend the Overall Testing Strategy 4 Marks

CO3 | Bloom's Level: K5 – Evaluate

Based on Tasks 1–10, recommend the overall testing strategy across all levels for AgriMart.

Discuss:

- **The activities at each level and how they connect**
- **Why this strategy is better than the alternatives**
- **The effort and schedule required**
- **Trade-offs, risks and limitations**
- **Whether the strategy will remain practical as AgriMart grows**

Your recommendation must be based on evidence from your own analysis.

Evidence Expected

A technically justified multi-level testing strategy supported by the analysis in earlier tasks.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Testing Strategy 8 Marks

CO3 | Bloom's Level: K5 – Evaluate

Changed Requirement

The department now requires AgriMart to exchange prices and orders with the national e-NAM trading platform.

A critical defect in M5 must be fixed and released as a hotfix within one week.

The e-NAM test environment is available only two hours a day.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO3 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed from the original situation**
- **Which units, interfaces and system tests are affected**
- **Why limited access to the e-NAM environment is important**
- **Why a hotfix creates regression risk**

Evidence Expected

A clear analysis of how the changed conditions affect the testing strategy.

B. Evaluation of the Original Strategy – 2 Marks

CO3 | Bloom's Level: K5 – Evaluate

Evaluate whether your recommended strategy remains appropriate.

Discuss:

- **Whether the integration order still works**
- **Whether the regression approach can handle the hotfix in one week**
- **Whether acceptance criteria need to change**

Evidence Expected

A technically supported evaluation of the original strategy under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO3 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the testing strategy.

If you modify or redesign it:

- **Describe the revised strategy for unit, integration, system and regression testing**
- **Explain how e-NAM will be simulated or scheduled**
- **Identify the minimum regression set for the hotfix**

If you retain the original strategy, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate response to the changed requirement with clear reasoning on test levels.

D. Evidence-Based Defence – 2 Marks

CO3 | Bloom's Level: K5 – Evaluate

Defend your final decision using evidence.

Use:

- **Module status and dependencies**

- **Changes and regression impact**
- **Environment availability**
- **User and business risk**
- **Schedule constraints**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final decision integrating test-level reasoning and the changed constraints.

TASK 1 – EVALUATE THE SCOPE OF UNIT TESTING

1. Definition of a Unit in AgriMart

A unit is the smallest independently testable part of a software application. In AgriMart, a unit should be a function, method or class that performs a specific operation rather than treating an entire screen as one unit.

Examples:

- M3 – Price Engine: Functions that calculate suggested prices, apply quality factors, validate farmer prices and check market-price API response time.
- M4 – Cart and Order: Functions that calculate cart totals, validate quantities, check order details and calculate cancellation deadlines.

2. Goals of Unit Testing

- Verify that each function works according to its requirements.
- Identify coding errors at an early stage.
- Check boundary conditions and invalid inputs.
- Ensure accurate price calculations and order processing.
- Make debugging easier before integrating modules.

3. Defects Unit Testing Can and Cannot Find

Defects it can find:

- Incorrect quality factor calculations.
- Wrong price limit validation.
- Invalid quantity calculations.
- Incorrect cancellation time calculations.
- Errors in handling null or invalid inputs.

Defects it cannot easily find:

- Communication failures between M3 and the market-price API.
- Integration problems between M3 and M4.
- Payment gateway failures.
- Problems caused by slow network connections.
- Complete application usability and compatibility issues.

4. Screen as Unit vs Function/Class as Unit

Screen as Unit:

- Tests the complete screen.
- May combine several functions.
- Makes it difficult to identify the exact location of defects.
- May require database and external service dependencies.

Function/Class as Unit:

- Tests individual operations independently.
- Makes defect identification easier.
- Supports automated testing.
- Allows boundary and exception testing.

5. Recommendation

I recommend using function/class as the unit of testing in AgriMart. For example, the price calculation and price validation functions in M3 should be tested separately. This provides better defect isolation, accuracy and test coverage. Screen-level testing can be performed later during system testing.

Therefore, function-level unit testing is more effective than treating each screen as a unit.

TASK 2 – EVALUATE UNIT TEST PLANNING

Unit Test Plan for M3 – Price Engine

1. Objectives:

- Verify accurate suggested price calculations.
- Validate all three quality grades.
- Check the ±15% farmer price limit.
- Verify API timeout and old-price handling.

2. Features to Be Tested:

- Grade A, Grade B and Grade C calculations.
- Market-price API response within and beyond 3 seconds.
- Last available price not older than 24 hours.
- Blocking listings when valid prices are unavailable.
- Minimum and maximum farmer price limits.

3. Testing Approach:

Use white-box unit testing with positive, negative, boundary and exception test cases. Mock the market-price API to obtain repeatable results.

4. Pass/Fail Criteria:

A test passes when the actual result matches the expected result. Incorrect calculations, invalid price acceptance or incorrect timeout handling are considered failures.

5. Resources and Time:

- One developer and one tester.
- A suitable unit testing framework.
- Mock API and prepared test data.
- Approximately one working day for test design, execution and defect correction.

6. Why Planning Is Skipped and Its Risks:

Unit test planning is sometimes skipped because of tight deadlines, lack of time or the belief that developers can test while coding. In AgriMart, skipping planning may cause incorrect farmer prices, financial losses and unfair transactions.

Conclusion: A specific test plan ensures that M3 is tested systematically before integration.

TASK 3 – DESIGN AND EVALUATE UNIT TESTS FOR THE PRICE ENGINE

1. Price Engine Rule

Suggested Price = Market Price × Quality Factor

- Grade A = 1.10
- Grade B = 1.00
- Grade C = 0.85

Farmer price must be within ±15% of the suggested price.

Example: Assume the market price is ₹100 per kg.

- Grade A = 100 × 1.10 = ₹110
- Grade B = 100 × 1.00 = ₹100
- Grade C = 100 × 0.85 = ₹85

2. Unit Test Cases

Test ID	Test Condition	Expected Result
UT01	Grade A, market price ₹100	Suggested price ₹110
UT02	Grade B, market price ₹100	Suggested price ₹100
UT03	Grade C, market price ₹100	Suggested price ₹85
UT04	Grade A farmer price ₹93.50	Accepted (lower limit)
UT05	Grade A farmer price ₹126.50	Accepted (upper limit)
UT06	Grade A farmer price ₹93.49	Rejected
UT07	Grade A farmer price ₹126.51	Rejected
UT08	API responds in 2 seconds	Latest market price used
UT09	API responds exactly in 3 seconds	Latest market price used
UT10	API responds in 4 seconds, cached price is 12 hours old	Cached price used
UT11	API fails, cached price is exactly 24 hours old	Cached price used
UT12	API fails, cached price is 25 hours old	Listing blocked
UT13	API fails and no cached price exists	Listing blocked
UT14	Market price is zero or negative	Invalid price rejected
UT15	Quality grade is invalid	Validation error displayed
UT16	Farmer enters a value outside ±15%	Price rejected

3. Boundary Value Calculations

For Grade A:

Suggested price = ₹110

- Minimum = 110 − 15% = ₹93.50
- Maximum = 110 + 15% = ₹126.50

For Grade B:

Suggested price = ₹100

- Minimum = ₹85
- Maximum = ₹115

For Grade C:

Suggested price = ₹85

- Minimum = ₹72.25
- Maximum = ₹97.75

The minimum and maximum values should be accepted. Values immediately outside these limits should be rejected.

4. Positive, Negative and Exception Testing

Positive Testing:

- Valid quality grades.
- Correct market price.
- Farmer price within the permitted range.
- API response within 3 seconds.

Negative Testing:

- Invalid quality grade.
- Negative market price.
- Farmer price outside ±15%.
- Market price older than 24 hours.

Exception Testing:

- API connection failure.
- API timeout.
- Missing market-price data.
- Null or invalid input values.

5. Impact of Change C1

Earlier Grade C quality factor was 0.80. It has now changed to 0.85.

For a market price of ₹100:

- Old suggested price = ₹100 × 0.80 = ₹80.
- New suggested price = ₹100 × 0.85 = ₹85.

Therefore, Grade C calculation tests, minimum and maximum boundary tests, and related price validation tests must be updated.

6. Evaluation of Test Sufficiency

The test set covers all three quality grades, price boundaries, API response times, cached price validity, invalid inputs and exceptions.

I would also check statement and branch coverage to ensure that all important calculation and validation paths are executed.

Conclusion: These tests provide good confidence that farmers receive accurate and fair suggested prices. Additional integration testing is still required to verify actual API communication.

TASK 4 – DESIGN AND EVALUATE THE TEST HARNESS

1. Test Harness for M3

A test harness is a testing environment that uses drivers, stubs and test data to test a module independently.

Driver:

- Calls the M3 Price Engine functions.
- Supplies market prices, quality grades and farmer prices.
- Checks actual results against expected results.

Stubs:

- M2 stub simulates produce listing details and quality grades.
- Market-price API stub simulates external market prices and API responses.

2. Simulation of API Conditions

The market-price API stub should simulate:

- Normal response: Returns a valid market price within 3 seconds.
- Slow response: Delays the response beyond 3 seconds.
- API failure: Returns connection error or timeout.
- Old price: Returns cached data older than 24 hours.
- Missing data: Returns an empty or invalid market price.

This helps test all M3 conditions without depending on the real API.

3. Storage of Test Data

Market Price	Grade	Expected Price
100	A	110
100	B	100
100	C	85

Test inputs and expected outputs should be stored in CSV or JSON files.

The test data should also contain API response time, cached price age and expected listing status.

4. Benefits and Limitations

Benefits:

- Repeatable testing.
- Faster defect identification.
- No dependency on daily market-price changes.
- Easy testing of API failures and timeout conditions.

Limitations:

- Stubs may not behave exactly like real services.
- Actual network and interface defects may remain undetected.

Risk Reduction:

Compare stub responses with the real API and perform integration testing in a controlled environment before release.

Conclusion: A driver and suitable stubs make M3 testing reliable, repeatable and independent of external service availability.

TASK 5 – EVALUATE INTEGRATION STRATEGIES

1. Big-Bang Integration

All modules are combined at once and tested together.

Evaluation: Not suitable because M5 and M6 have not started, while M4 and M7 are still in progress. Defects would be difficult to identify.

2. Top-Down Integration

Testing starts from higher-level modules and gradually integrates lower-level modules using stubs.

Evaluation: It can test the application flow early, but incomplete modules such as M5 and M6 require many stubs.

3. Bottom-Up Integration

Testing starts with lower-level modules and gradually integrates higher-level modules using drivers.

Evaluation: Suitable for testing completed modules such as M1, M2 and M3 independently. However, complete application workflows are tested later.

4. Sandwich Integration

Combines top-down and bottom-up approaches. Different module groups are tested in parallel and joined later.

Evaluation: It is useful because AgriMart has modules at different completion stages. Completed modules can be tested while unfinished modules are developed.

5. Recommended Strategy

I recommend a risk-based sandwich integration strategy.

- Test M1, M2 and M3 using bottom-up integration.
- Use stubs for unfinished M5 and M6.
- Integrate M4 and M7 as they become ready.
- Test the complete application workflow after all modules are connected.

External Aadhaar e-KYC, market-price API, UPI gateway and transport partner API should initially be simulated using stubs and later tested with actual services.

Conclusion: Sandwich integration is the best choice because it supports parallel testing, handles incomplete modules and identifies important interface defects early.

TASK 6 – CONSTRUCT AND EVALUATE THE INTEGRATION TEST PLAN

1. Integration Order

Step 1: M1 + M2

- Verify that registered farmers can create produce listings.
- Use an M1 stub if required.
- Entry: M1 and M2 unit tests passed.
- Exit: Valid registration details reach M2 correctly.

Step 2: M2 + M3

- Verify that produce details and quality grades reach the Price Engine.
- Check that M3 returns the correct suggested price.
- Use a market-price API stub.
- Entry: M2 and M3 unit tests passed.
- Exit: Correct produce and price information is returned.

Step 3: M3 + M4

- Verify that the suggested price reaches the cart and order module.
- Test price changes, invalid prices and total calculations.
- Entry: M3 and M4 unit tests passed.
- Exit: Correct prices and quantities are used in orders.

Step 4: M4 + M5

- Verify that order details and payable amounts reach the UPI gateway.
- Use a UPI stub until the actual module is available.
- Entry: M4 tests passed and M5 is completed.
- Exit: Successful and failed payments are handled correctly.

Step 5: M4 + M6

- Verify that confirmed orders are sent to the logistics module.
- Use a transport partner API stub.
- Entry: M4 and M6 tests passed.
- Exit: Pickup and delivery details are correctly transferred.

Step 6: M1–M7 Integration

- Verify SMS and Tamil voice notifications.
- Check the complete farmer-to-buyer workflow.
- Entry: All modules and interfaces pass integration tests.
- Exit: No critical interface defects remain.

2. Important Interface Test Cases

M3–M4:

- Correct suggested price is transferred to the cart.
- Updated price is reflected in the order.

- Invalid farmer price is rejected.
- Total amount is calculated correctly.

M4–M5:

- Correct order amount reaches UPI.
- Successful payment updates order status.
- Failed payment does not create a paid order.
- Duplicate payment requests do not cause duplicate charges.

Conclusion: Step-by-step integration with suitable stubs reduces interface defects and allows testing to continue despite incomplete modules.

TASK 7 – EVALUATE SYSTEM TESTING NEEDS

System testing checks the complete AgriMart application after integrating all modules.

1. Essential Types of System Testing

1. Functional Testing

Verifies whether all application features work correctly.

Scenario: A farmer registers, lists produce, receives a suggested price and completes an order.

Risk: Incorrect transactions or missing functionality.

2. Performance and Load Testing

Checks application performance under expected user load.

Scenario: Simulate 25,000 farmers and 3,000 buyers using realistic concurrent-user workloads.

Risk: Slow response, server overload and application crashes.

3. Usability Testing

Checks whether users can easily understand and operate the application.

Scenario: Ask farmers with limited literacy to register, list produce and understand Tamil voice prompts.

Risk: Farmers may enter incorrect details or fail to complete transactions.

4. Security Testing

Checks whether user data and transactions are protected.

Scenario: Test unauthorized access, password security, payment information protection and secure API communication.

Risk: Data theft, fraud and unauthorized transactions.

5. Compatibility Testing

Checks whether the application works on different devices and environments.

Scenario: Test on low-cost Android phones, different screen sizes and regional language settings.

Risk: Application failure on farmers' actual devices.

6. Recovery Testing

Checks whether the application recovers after failures.

Scenario: Disconnect the network during payment and verify that the application does not create duplicate orders after reconnecting.

Risk: Data loss and duplicate transactions.

2. Testing Priority

Highest Priority:

- Functional testing.
- Security testing.
- Performance testing.
- Recovery testing.

Important Priority:

- Usability testing.
- Compatibility testing.

All six types are necessary before production. If time is limited, reduce repeated low-risk compatibility combinations and use automated testing for routine checks. Do not skip security, payment recovery or core functional testing.

Conclusion: AgriMart must be tested under realistic rural conditions to ensure reliability, security and usability.

TASK 8 – EVALUATE REGRESSION TESTING

Regression testing verifies that recent software changes have not affected existing functionality.

1. Types of Regression Testing

Retest-All: Executes the complete test suite after every change. It provides broad coverage but requires more time.

Selective Regression: Executes only tests related to the modified modules and their dependencies.

Risk-Based Regression: Prioritizes tests based on the possibility and impact of failure.

I recommend selective and risk-based regression testing for AgriMart.

2. Regression Tests for Each Change

C1 – Grade C Quality Factor Changed from 0.80 to 0.85 (M3)

- Retest Grade C price calculation.
- Retest ±15% price boundaries.
- Retest produce listing prices.
- Check M4 cart and order totals.

Indirectly affected modules: M4 and M7 because they use or communicate price information.

C2 – Cancellation Window Reduced from 2 Hours to 1 Hour (M4)

- Test cancellation within one hour.
- Test cancellation after one hour.
- Test cancellation exactly at the one-hour boundary.
- Verify payment refund and order status.

Indirectly affected modules: M5, M6 and M7 because cancellation can affect payment, logistics and notifications.

C3 – New Tamil Voice Prompts Added (M7)

- Verify Tamil voice prompts.
- Check pronunciation and correct messages.
- Test voice notifications on low-cost phones.
- Verify existing SMS notifications.

Indirectly affected modules: M1–M6 because notifications may be triggered by registration, listing, orders, payment and logistics.

C4 – Duplicate Farmer Registration Fix (M1)

- Register a new farmer.
- Attempt registration using the same mobile number.
- Verify duplicate registration is rejected.
- Verify existing farmer login still works.

Indirectly affected modules: M2, M3, M4 and M7 because farmer identity is required for listings, pricing, orders and notifications.

3. Automation in Regression Testing

Automation tools can execute repeated test cases after every code change.

- Automate price calculations and boundary tests.
- Automate registration and duplicate mobile checks.
- Automate order cancellation tests.
- Automate notification and payment workflow checks.
- Generate test reports to identify failures quickly.

4. Recommendation

Risk-based selective regression testing is suitable because it focuses on modified modules and indirectly affected dependencies while saving execution time.

However, a complete regression suite should be executed before major production releases.

Conclusion: Regression testing protects existing functionality and prevents new changes from introducing unexpected defects.

TASK 9 – EVALUATE ALPHA AND BETA TESTING

1. Alpha Testing

Alpha testing is performed internally before releasing the application to real users.

Participants:

- Developers.
- Internal testing team.
- Agriculture department representatives.

Location: Controlled testing environment.

Duration: Approximately one week.

Activities:

- Test farmer registration and produce listing.
- Verify price calculations and payment workflows.
- Check Tamil language and voice notifications.
- Identify usability and functional defects.

2. Beta Testing

Beta testing is performed by selected real users in actual working environments before public release.

Participants:

- 30–50 farmers.
- 10–15 buyers.
- Selected agricultural field representatives.

Location: Different rural areas with slow networks and different mobile devices.

Duration: Approximately two weeks.

Activities:

- Register and list agricultural products.
- Allow buyers to browse and place test orders.
- Check application performance under actual network conditions.
- Collect feedback about regional language and usability.

3. Feedback Collection

Farmers with limited literacy may find written feedback difficult. Therefore:

- Provide Tamil voice feedback options.
- Conduct short interviews.
- Use simple icons and visual feedback.
- Allow field representatives to record user problems.
- Collect crash reports and application usage information.

4. Difference and Importance

Alpha testing identifies defects in a controlled environment, while beta testing identifies real-world problems that may not appear during internal testing.

Conclusion: Both alpha and beta testing are important to ensure AgriMart is reliable and suitable for farmers and buyers before launch.

TASK 10 – EVALUATE ACCEPTANCE TESTING

Acceptance testing determines whether AgriMart satisfies the agreed requirements and is ready for official release.

1. Measurable Acceptance Criteria

The agriculture department should agree on the following criteria before testing:

1. All critical farmer registration and produce listing functions must work correctly.
2. All three quality grades must produce accurate suggested prices.

3. Farmer prices outside ±15% must be rejected.
4. API timeout and 24-hour cached price rules must work correctly.
5. Successful payments must update order status correctly.
6. No unresolved critical or high-severity security defects should remain.
7. The application must support the agreed regional language and Tamil voice prompts.
8. The application must pass the agreed performance workload representing the expected 25,000 farmers and 3,000 buyers.
9. Order cancellation must follow the one-hour rule.
10. No critical data loss or duplicate payment defects should remain.

2. Acceptance Test Scenarios

Farmer:

- Register using mobile number and e-KYC.
- List produce with quality grade.
- Receive suggested price.
- Set a price within the permitted limit.

Buyer:

- Browse available produce.
- Add products to cart.
- Place an order.
- Complete UPI payment.
- Cancel an order within the permitted period.

Administrator:

- View farmer and buyer details.
- Monitor produce listings and orders.
- Verify notifications and transaction records.

- Check system reports.

3. Who Performs Acceptance Testing?

- Selected farmers and buyers perform user acceptance testing.
- Agriculture department representatives verify business requirements.
- Independent testers support test execution and defect reporting.
- The department's authorized project manager or acceptance committee provides final sign-off.

4. If Acceptance Criteria Are Not Met

- Record all failed test cases.
- Classify defects based on severity.
- Return the defects to the development team.
- Correct critical and high-severity defects.
- Perform regression testing.
- Repeat the failed acceptance tests.
- Obtain formal approval only after agreed criteria are satisfied.

Conclusion: Formal acceptance testing ensures that AgriMart meets the department's requirements before vendor payment and production release.

TASK 11 – RECOMMEND AND DEFEND THE OVERALL TESTING STRATEGY

1. Overall Testing Strategy

I recommend a risk-based testing strategy combining unit, integration, system, regression and acceptance testing.

Unit Testing:

- Test individual functions and classes.
- Focus on price calculation, validation, registration and order calculations.
- Use positive, negative and boundary test cases.

Integration Testing:

- Integrate M1, M2, M3 and M4 step by step.
- Use stubs for incomplete modules and external services.
- Verify data transfer and communication between modules.

System Testing:

- Test the complete application.
- Perform functional, performance, security, usability, compatibility and recovery testing.
- Use realistic rural network and device conditions.

Regression Testing:

- Test modified modules and dependent modules.
- Prioritize financial, registration and order-related functions.
- Automate repeated test cases.

Acceptance Testing:

- Allow selected farmers, buyers and department representatives to validate the application.
- Obtain formal approval before production release.

2. Why This Strategy Is Better

- Finds defects early through unit testing.
- Identifies interface problems through integration testing.
- Checks complete application behaviour through system testing.
- Prevents old defects from returning through regression testing.
- Confirms business requirements through acceptance testing.

3. Effort and Schedule

A

Activity	Estimated Time
Unit testing	3 days
Integration testing	4 days
System testing	4 days
Regression testing	2 days
Acceptance testing	2 days

proposed testing schedule is:

Some activities can be performed in parallel to reduce the overall schedule. Actual duration depends on module readiness and defect correction.

4. Trade-offs and Risks

- Automation requires initial setup time.
- Stubs may not represent real external services perfectly.
- Testing all device combinations requires additional effort.
- Limited time may affect complete regression coverage.

These risks can be reduced through prioritization, automation and real-service integration checks.

5. Future Scalability

As AgriMart grows, automated testing, performance testing and continuous regression testing can be expanded to support additional farmers, buyers, crops and regions.

Conclusion: A risk-based multi-level testing strategy provides better quality, early defect detection and reliable application performance.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE TESTING STRATEGY

A. IMPACT ANALYSIS

1. What Has Changed?

In the original situation, AgriMart depended on Aadhaar e-KYC, market-price API, UPI gateway and transport partner API.

Now, the department requires integration with the national e-NAM trading platform. Additionally, a critical M5 payment defect must be fixed and released within one week.

The e-NAM testing environment is available for only two hours per day.

2. Affected Units, Interfaces and System Tests

Affected Units:

- M3 price exchange functions.
- M4 order processing functions.
- M5 payment functions.
- New e-NAM data exchange functions.

Affected Interfaces:

- M3 with e-NAM for price exchange.
- M4 with e-NAM for order exchange.
- M4 with M5 for payment processing.

Affected System Tests:

- Price synchronization.
- Order data exchange.
- Payment success and failure.
- Duplicate transaction prevention.
- Recovery after network failure.

3. Importance of Limited e-NAM Environment Access

The e-NAM environment is available for only two hours daily. Therefore, testing time is limited and must be planned carefully.

Mock services and stubs should be used for preliminary testing. Actual e-NAM testing should be scheduled for the available two-hour window.

4. Hotfix Regression Risk

A hotfix is a quick software update used to correct a critical defect.

However, fixing M5 may introduce new defects in payment processing, order status, refunds or notifications.

Therefore, focused regression testing is essential before release.

B. EVALUATION OF THE ORIGINAL STRATEGY

1. Integration Order

The original sandwich integration strategy is still useful, but it must be modified to include e-NAM.

The new integration should focus on:

- M3 and e-NAM price exchange.
- M4 and e-NAM order exchange.
- M4 and M5 payment processing.

2. Regression Approach

The original selective and risk-based regression approach is suitable, but the critical M5 hotfix requires immediate priority.

Payment-related tests and dependent order, logistics and notification tests must be executed before release.

3. Acceptance Criteria

Acceptance criteria must be updated to include:

- Correct e-NAM price exchange.
- Correct order synchronization.
- No duplicate or missing transactions.

- Successful payment processing.
- No critical defects in the hotfix.

Evaluation: The original strategy is still appropriate as a foundation, but it cannot be used without modification because of the new external platform and urgent release requirement.

C. ADAPTATION OR REDESIGN

Recommended Decision: MODIFY THE ORIGINAL STRATEGY

I recommend modifying the original risk-based sandwich integration strategy instead of completely redesigning it.

1. Revised Unit Testing

- Test M5 payment processing functions.
- Test transaction validation and duplicate payment prevention.
- Test M3 e-NAM price conversion and validation functions.
- Test M4 order exchange functions.

2. Revised Integration Testing

- Use an e-NAM stub to simulate price and order exchange.
- Test M3 and M4 interfaces independently.
- Schedule actual e-NAM integration during the available two-hour period.
- Test M4 and M5 after fixing the payment defect.

3. Revised System Testing

- Verify the complete farmer-to-buyer workflow.
- Check price and order synchronization with e-NAM.
- Test payment success, failure and recovery.
- Check application performance and security.

4. Revised Regression Testing

The minimum hotfix regression set should include:

1. Successful UPI payment.
2. Failed UPI payment.
3. Duplicate payment prevention.
4. Correct order status after payment.
5. Refund and cancellation processing.
6. Logistics notification after confirmed payment.
7. SMS and Tamil voice notifications.
8. Order history and transaction records.

5. Seven-Day Hotfix Schedule

Day	**Activity**
Day 1	Analyse M5 defect and identify affected modules
Day 2	Fix defect and perform unit testing
Day 3	Perform M4–M5 integration testing
Day 4	Execute payment regression tests
Day 5	Test e-NAM interfaces using stubs and scheduled environment access
Day 6	Perform focused system and security testing
Day 7	Final verification and hotfix release

D. EVIDENCE-BASED DEFENCE

The decision to modify the original strategy is supported by the following evidence:

1. Module Status and Dependencies:

M5 was originally not started, while M4 was still in progress. Payment testing must therefore receive priority before integrating the complete application.

2. Changes and Regression Impact:

The M5 hotfix may affect payment status, order processing, cancellation, refunds and notifications. Therefore, dependent modules must also be tested.

3. Environment Availability:

The e-NAM environment is available for only two hours per day. Using stubs for early testing and scheduling actual integration testing reduces dependency on limited access.

4. User and Business Risk:

Incorrect prices, failed payments and duplicate orders can cause financial losses to farmers and buyers. These functions require high-priority testing.

5. Schedule Constraints:

The hotfix must be released within one week. Therefore, risk-based testing, automated regression tests and parallel activities are necessary.

Final Recommendation

I recommend modifying the existing risk-based sandwich integration strategy. Unit testing should validate individual payment and price functions, integration testing should verify e-NAM and payment interfaces, system testing should validate complete business workflows, and regression testing should focus on critical payment and order functions.

This modified strategy balances software quality, business risk, limited external environment access and the one-week release deadline.

Conclusion: Modifying the original strategy is the most practical decision because it addresses the new e-NAM requirement and critical payment defect without delaying the entire AgriMart project.

MARK DISTRIBUTION

Task	Marks
Task 1 – Evaluate the Scope of Unit Testing	5
Task 2 – Evaluate Unit Test Planning	2
Task 3 – Design and Evaluate Unit Tests for the Price Engine	6
Task 4 – Design and Evaluate the Test Harness	4
Task 5 – Evaluate Integration Strategies	3
Task 6 – Construct and Evaluate the Integration Test Plan	3
Task 7 – Evaluate System Testing Needs	4
Task 8 – Evaluate Regression Testing	4
Task 9 – Evaluate Alpha and Beta Testing	3
Task 10 – Evaluate Acceptance Testing	4
Task 11 – Recommend and Defend the Overall Testing Strategy	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50